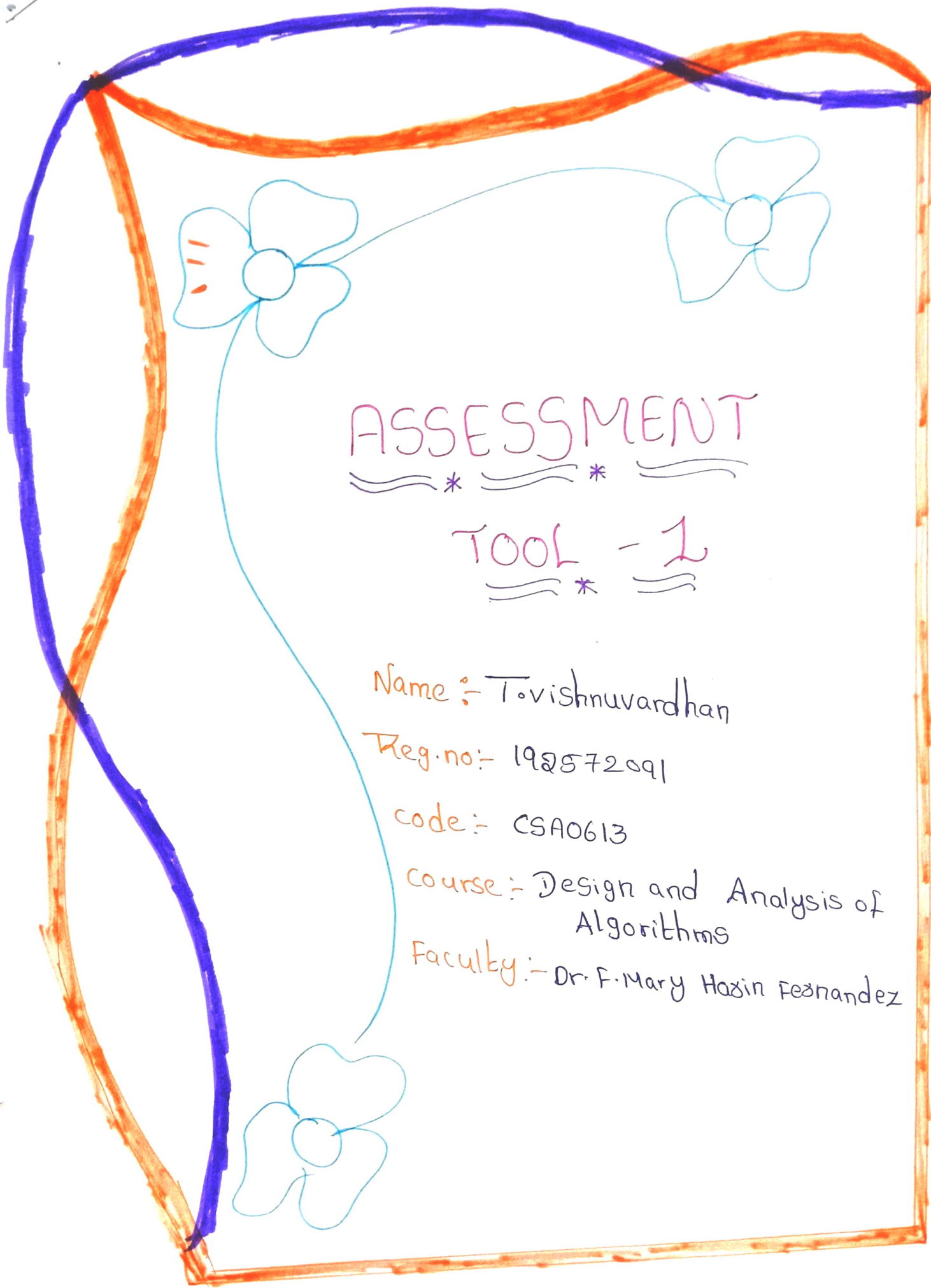




ASSESSMENT

TOOL - 1

Name :- Tovishnuvardhan

Reg.no:- 192572091

code:- CSA0613

course:- Design and Analysis of Algorithms

Faculty:- Dr. F. Mary Hazin Fernandez

Linear Search - Single Match in Middle position

39. Given the unsorted dataset [14, 28, 38, 47, 59, 63, 71], apply Linear search to find the element 47. Clearly interpret the problem and explain the sequential search process step-by-step. Count the no of comparisons needed before the target is found.

Analyze the best-case, average case, and worst-case time complexity. Verify the correctness of the result. Interpret the efficiency of linear search when the target lies near the middle of list.

A)

Problem understanding:-

Problem Interpretation:- we are giving an unsorted dataset. we have to find the element 47 using linear search. Linear search checks each element one by one from the beginning until the target element is found or the list ends.

Input:-

- Dataset : [14, 28, 35, 47, 59, 63, 71]
- Target element : 47

Requirements:-

- * Perform Linear search step by step
- * Count the number of Requirements
- * Analyze best case, average case and worst case

- * verify the correctness of the Result
- * Interpret the efficiency when the target lies near the middle

2. Method selection:-

Selected Method: Linear search

Justification:-

- The dataset is unsorted
- Linear search doesnot require sorting
- It checks elements sequentially
- It is simple to implement
- It guarantees the element will be found if it exists

Algorithm (Linear Search)

1. start from the first element
2. Compare the current element with the target
3. If equal, return to index and stop
4. otherwise, move to the next element
5. Repeat until the target is found or to be list ends

3. Solution Process

Dataset :-

0	1	2	3	4	5	6
14	28	35	47	59	63	71

Step by step Linear search

Step	Index	Current Element	Comparison ($A[i] == 47$)	Result	Comparison count
1	0	14	$14 == 47$	False	1
2	1	28	$28 == 47$	False	2
3	2	35	$35 == 47$	False	3
4	3	47	$47 == 47$	True	4

Final Result:- Target found

Target element:- 47

Found At Index:- 3

Total comparison:- 4

Step:1

Compare 14 with 47

14	28	35	47	59	63	71
----	----	----	----	----	----	----

Step:2

compare 28 with 47

14	28	35	47	59	63	71
----	----	----	----	----	----	----

Step:3

compare 35 with 47

14	28	35	47	59	63	71
----	----	----	----	----	----	----

Step:4

14	28	35	47	59	63	71
----	----	----	----	----	----	----

Compare 47 with 47

Equal

4. Accuracy of calculation:-

Number of comparisons:

1st comparison: $14 \neq 47 \rightarrow$ Not Equal

2nd comparison: $28 \neq 47 \rightarrow$ Not Equal

3rd comparison: $35 \neq 47 \rightarrow$ Not Equal

4th comparison: $47 = 47 \rightarrow$ Equal

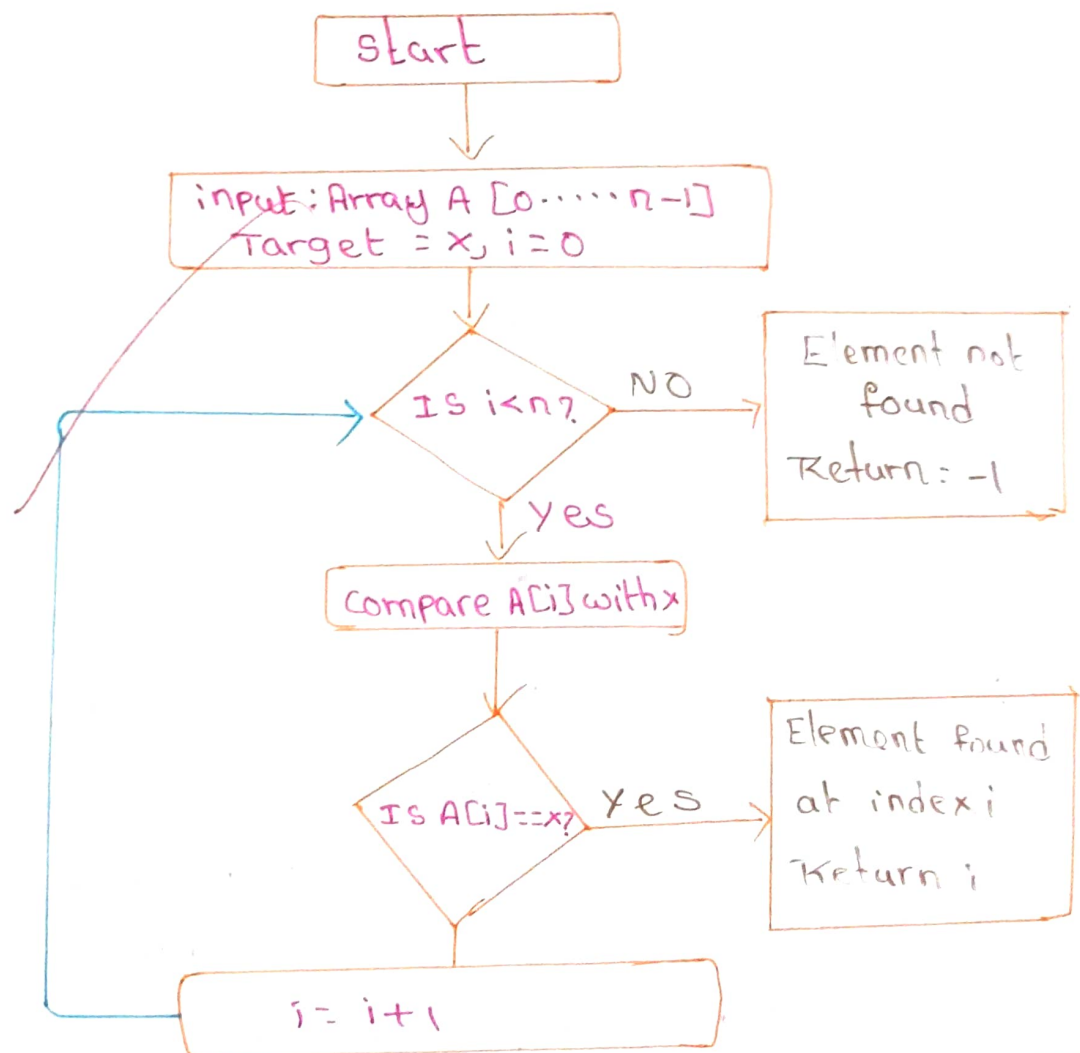
Total comparisons: 4

5. Time complexity Analysis

case	condition	No of Cmp	Time complexity
Best case	Target is the first element	1	$O(1)$
Average case	Target is middle of Average comparison = $n/2$	$n/2$	$O(n)$
Worst case	Target is the last Element or not present	n	$O(n)$

where n = number of Elements in the list

6. Flow chart of Linear Search:



PSEUDOCODE

LinearSearch(A, n, x)

```
i ← 0
while i < n do
    if A[i] == x then
        return i
    end if
    i ← i + 1
end while
return -1
```

Interpretation of Result:

The target element 47 is located near the middle of the list. Therefore, Linear search required 12 comparisons, which is equal to the average case $\frac{(n+1)}{2} = 12$. Linear search works by checking element sequentially so its time complexity is $O(n)$ for the average and worst cases.

Summary table

Algorithm used	Linear search
Dataset	[14, 28, 35, 47, 59, 63, 71]
Target element	47
Found at index	3
Total comparison	4
Best case	$O(1)$
Average case	$O(n)$
Worst case	$O(n)$
Result	Successfully

Conclusion:

Linear Search correctly found the target element 47 at the index 3 after 4 comparisons.

It is easy to implement works on unsorted data. Its performance is acceptable when the target is near the middle, but the time complexity grows with the input size.